

# SOLID Principles & Spring Boot Notes

---

## S: Single Responsibility Principle (SRP)

### Rule

**One class = One responsibility** (A class should have only one reason to change).

### Bad Example

```
@Service
public class UserService {
    public User getUser(Long id) {
        return userRepository.findById(id).get();
    }
    public void sendEmail(User user) {
        // email logic
    }
    public void generatePdf(User user) {
        // pdf logic
    }
}
```

**What is the purpose of UserService?** User management? Email dispatching? PDF generation? It handles everything. This is a clear SRP violation.

### The Problem

If your email provider changes, you must edit UserService. If your PDF library or formatting changes, you must edit UserService. One class changes for multiple, unrelated reasons.

### Better Design

```
@Service
public class UserService {
    public User getUser(Long id) {
        return userRepository.findById(id).get();
    }
}

@Service
public class EmailService {
    public void sendEmail(User user) { /* email logic */ }
}

@Service
public class PdfService {
    public void generatePdf(User user) { /* pdf logic */ }
}
```

Now, every class has a single, well-defined purpose.

### Real Spring Boot Example

Spring Boot architecture enforces SRP by default through layer separation:

- @RestController: Handles HTTP requests and responses.
- @Service: Handles core business logic.
- @Repository: Handles database interactions and persistence.

---

## O: Open/Closed Principle (OCP)

### Rule

**Open for extension, closed for modification.** Add new features by writing new code, not by altering existing code.

### The Problem (Payment System)

## L: Liskov Substitution Principle (LSP)

### Rule

**Subclasses must be substitutable for their base types without altering program correctness.**

### Bad Example

```
interface Bird {
    void fly();
}

class Sparrow implements Bird {
    public void fly() { /* Sparrow flies */ }
}

class Penguin implements Bird {
    public void fly() {
        // Penguins cannot fly
        throw new UnsupportedOperationException();
    }
}
```

Throwing an exception breaks caller expectations because a Penguin cannot fulfill the Bird contract. This violates LSP.

### Better Design

```
interface Bird {
    // Shared bird attributes or behaviors
}

interface FlyingBird extends Bird {
    void fly();
}

class Sparrow implements FlyingBird {
    public void fly() { /* implementation */ }
}

class Penguin implements Bird {
    // No fly method forced here
}
```

### Real Spring Boot Example

When using JpaRepository:

- save()
- findById()
- delete()

Regardless of the underlying implementation or database engine (Hibernate, H2, PostgreSQL), callers expect identical contract behavior across all repository beans.

---

## I: Interface Segregation Principle (ISP)

### Rule

**Clients should not be forced to depend on methods they do not use.** Prefer many small, focused interfaces over one large, bloated interface.

### Bad Example

```
public interface Worker {
    void work();
    void eat();
    void sleep();
}
```

## D: Dependency Inversion Principle (DIP)

### Rule

1. High-level modules should not depend on low-level modules. Both should depend on abstractions (interfaces).
2. Abstractions should not depend on details. Details should depend on abstractions.

### Bad Example

```
@Service
public class OrderService {
    // High-level service tightly coupled to a concrete low-level implementation
    private MySQLRepository repository = new MySQLRepository();
}
```

Switching from MySQL to MongoDB requires rewriting the OrderService class directly.

### Better Design

```
public interface OrderRepository {
    void save();
}

@Repository
public class MySQLRepository implements OrderRepository {
    public void save() { /* MySQL insert */ }
}

@Repository
public class MongoRepository implements OrderRepository {
    public void save() { /* Mongo insert */ }
}

@Service
public class OrderService {
    private final OrderRepository repository;

    public OrderService(OrderRepository repository) {
        this.repository = repository;
    }
}
```

OrderService no longer cares whether the storage layer is MySQL, MongoDB, or an in-memory database.

### Real Spring Boot Example

Spring's core mechanism—Inversion of Control (IoC) and Dependency Injection (DI)—is a direct implementation of DIP:

```
@Service
public class OrderService {
    private final OrderRepository repository;

    @Autowired
    public OrderService(OrderRepository repository) {
        this.repository = repository;
    }
}
```

Spring injects the concrete bean dynamically at runtime; the service only references the OrderRepository interface.

## 2-Minute Interview Pitch: "How Does Spring Boot Follow SOLID Principles?"

| Principle  | How Spring Boot Implements It                                                                                                      |
|------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>SRP</b> | Distinct architectural layers: @RestController handles HTTP, @Service handles business logic, and @Repository handles data access. |